Национальная научно-образовательная корпорация ИТМО

Факультет программной инженерии и компьютерной техники

ЛАБОРАТОРНАЯ РАБОТА №3-4

По дисциплине

«Программирование»

Вариант 3109095.8

Выполнил:

Студент группы P3109

Халкечев Дмитрий Артурович

Преподаватель:

Наумова Надежда Александровна

Санкт-Петербург, 2025

Цель работы: на простом примере разобраться с основными концепциями ООП и научиться использовать их в программах.

Задание:

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Описание предметной области, по которой должна быть построена объектная модель:

Чтобы разделить персик, нужен был нож: и Малыш побежал в кухню. А когда он вернулся с ножом, Карлсон исчез. Но Малыш тут же услышал, что из-под стола доносилось чавканье и причмокивание, словно кто-то торопливо ел что-то очень сочное. Когда Карлсон вылез из-под стола, персиковый сок стекал у него с подбородка. Он протянул свою пухлую ручку и сунул Малышу большую шершавую темно-красную косточку. Но Малыш, если бы и захотел, не успел бы ничего ответить, потому что в мгновение ока Карлсон очутился у окна, где на подоконнике стоял горшок с бегонией и схватил цветок за стебель. Но было уже поздно: Карлсон вырвал бегонию с корнем и вышвырнул в окно. Карлсон успел тем временем ткнуть косточку в горшок и теперь энергично засыпал ее землей. Малыш об этом не подумал, поэтому он не знал, что ответить. Спорить с Карлсоном было нелегко, особенно когда он бывал в настроении спорить. Но, к счастью, настроение у него менялось каждые пятнадцать минут. Вот и теперь он вдруг издал какой-то странный, но явно радостный звук, похожий на кудахтанье. Этого Малыш прежде не знал. Глаза Карлсона заблестели. Он схватил пакет и со всех ног побежал в ванную комнату. Малыш, раздираемый любопытством, бросился за ним. Ему очень хотелось узнать, как делают самый громкий в мире хлюп.

Этапы выполнения работы:

1. Получить вариант
2. Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
3. Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
4. Согласовать диаграмму классов и сценарий с преподавателем.
5. Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
6. Для одного из методов класса персонажа (или объекта), согласованного с преподавателем, создать промпты для 2-3 ИИ-ассистентов (Gigachat, DeepSeek, Qwen, или др.), получить от них код метода, **реализовать метод самостоятельно**, и включить в отчет сравнительный анализ кода, написанного ИИ-ассистентами, и своего

варианта. Оценить качество кода, полученного от ИИ-ассистентов, указать на возможные недоработки.
7. Продемонстрировать выполнение программы на сервере `helios`.
8. Ответить на контрольные вопросы и выполнить дополнительное задание.

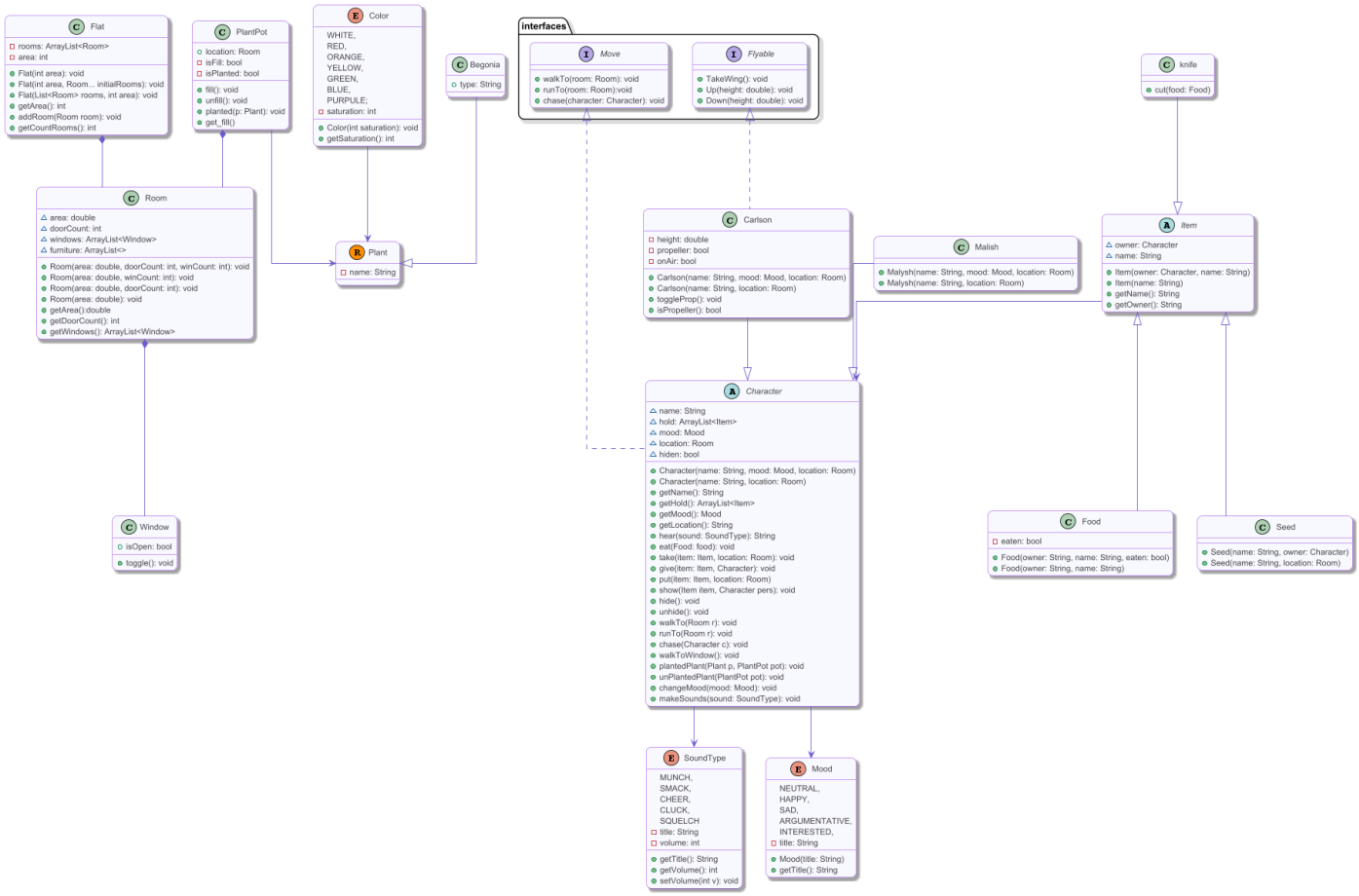
Текст, выводящийся в результате выполнения программы не обязан дословно повторять текст, полученный в исходном задании. Также не обязательно реализовывать грамматическое согласование форм и падежей слов выводимого текста.

Стоит отметить, что цель разработки объектной модели состоит не в выводе текста, а в эмуляции объектов предметной области, а именно их состояния (поля) и поведения (методы). Методы в разработанных классах должны изменять состояние объектов, а выводимый текст должен являться побочным эффектом, отражающим эти изменения.

Требования к объектной модели, сценарию и программе:

1. В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.
2. Объектная модель должна реализовывать основные принципе ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.
3. Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).
4. Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:
    - абстрактный класс как минимум с одним абстрактным методом;
    - интерфейс;
    - перечисление (enum);
    - запись (record);
    - массив или ArrayList для хранения однотипных объектов;
    - проверяемое исключение.
5. В созданных классах основных персонажей и предметов должны быть корректно переопределены методы `equals()`, `hashCode()` и `toString()`. Для классов-исключений необходимо переопределить метод `getMessage()`.
6. Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно unchecked исключение (можно свое, можно из стандартной библиотеки).
7. При необходимости можно добавить внутренние, локальные и анонимные классы.

Диаграмма классов реализованной объектной модели:



Код:

https://gitlab.se.ifmo.ru/DmitryHalk/lab3-4prog

Результат работы:

https://gitlab.se.ifmo.ru/DmitryHalk/lab3-4prog/-/blob/master/Result

Сравнение реализации метода:

Промпт: У тебя есть класс Character с такими полями: protected String name;

protected ArrayList<Item> hold = new ArrayList<>();

protected Mood mood;

protected Room location;

protected boolean hiden;

а также класс public class Room {

protected String name;

protected double area;

protected int doorCount;

protected ArrayList<Window> windows;

с соответсвтующими конструктарами, геттерами и сеттерами, ну и имеется класс Flat в котором хранятся все комнаты

реализуй метод walkTo() для Character - для перемещения в другое помещение внутри квартиры. Не добавляй новые поля от себя

DeepSeek:

```
public boolean walkTo(Room targetRoom) {
    // Проверяем, что персонаж не скрыт
    if (hiden) {
      System.out.println(name + " скрыт и не может перемещаться!");
      return false;
    }
    // Проверяем, что целевая комната существует
    if (targetRoom == null) {
      System.out.println(name + " пытается пойти в несуществующую комнату!");
      return false;
    }
    // Проверяем, не пытается ли персонаж остаться в той же комнате
    if (targetRoom.equals(location)) {
      System.out.println(name + " уже находится в " + targetRoom.getName());
      return false;
    }
    // Сохраняем текущую комнату для сообщения
    Room previousRoom = location;

    // Перемещаем персонажа
    location = targetRoom;

    // Обновляем информацию о персонаже в комнатах (если такой функционал есть)
    updateRoomPresence(previousRoom, targetRoom);

    // Логируем перемещение
    System.out.println(name + " перешёл из " +
            (previousRoom != null ? previousRoom.getName() : "ниоткуда") +
            " в " + targetRoom.getName());

    // Обновляем настроение при перемещении (если это предусмотрено в классе Mood)
    updateMoodAfterMoving(previousRoom, targetRoom);

    return true;
```

ChatGPT 5.2

```
public void walkTo(Room target, Flat flat) {
```

```
    if (target == null) {
        throw new IllegalArgumentException("Целевая комната не может быть null");
    }
    if (flat == null || flat.getRooms() == null) {
        throw new IllegalStateException("Квартира не инициализирована");
    }
    if (!flat.getRooms().contains(target)) {
        throw new IllegalArgumentException(
            "Комната " + target.getName() + " не принадлежит данной квартире"
        );
    }
    if (target.equals(location)) {
        return; // уже в этой комнате — ничего не делаем
    }
    this.location = target;
}
```

Сравнение результатов:

Нейросети справились хорошо, однако из-за невозможности получить доступ сразу ко всем файлам, DeepSeek меняет возвращаемое значение с void на Boolean, что приведёт к ошибке так как метод реализует интерфейс, плюс к этому DeepSeek добавляет методы, которые не существуют, от себя, при этом без реализации что усложняет написание кода. В этом плане ChatGPT справился лучше, метод не изменяет сигнатуру, не добавляет лишних зависимостей и использует исключения для обработки некорректных действий. В то же время в моей реализации применяются проверяемые исключения, что упрощает контроль и обработку ошибок на уровне вызывающего кода.

Вывод:

В этой лабораторной работе мы создали объектно-ориентированную модель персонажей и объектов на Java и написали программу, которая имитирует их поведение. Модель использует основные принципы ООП и SOLID, а также включает абстрактные классы, интерфейсы, enum, record, коллекции и исключения. А также сравнили собственную реализацию с реализацией от нейросетей.